RNNs and LSTMs

ECON 8310: Business Forecasting — Lecture 9, Part 1

Department of Economics

University of Nebraska at Omaha

October 13, 2026

Lecture Outline

1. Where the Last Two Lectures Ran Out
2. The Vanilla RNN
3. LSTM: Long Short-Term Memory
4. In PyTorch, and Does It Work
5. Key Takeaways

Where the Last Two Lectures Ran Out

Two architectures, two different ways of failing to see time.

Two Architectures That Cannot See Time

Lecture 7 gave you a feedforward network. It treats each row independently — everything it knows about *when* something happened, you encoded by hand as a lag column. Shuffle the features and the model is unchanged.

Lecture 8 gave you a 1D CNN, which does see local shape. But its **receptive field** is fixed by architecture: stacked width-5 and width-3 filters reach about seven weeks back, and no reasonable stack of small filters reaches an annual lag. Worse, pooling deliberately discarded *where* in the window each pattern occurred — and we measured that costing about 500 RMSE.

> Both architectures share one assumption: the input is a **fixed-size block** to be processed all at once. Neither has any notion of reading a sequence *in order* and carrying something forward.

That is the gap today closes.

The Recurrent Idea: Carry State Forward

A recurrent network reads the sequence one step at a time and maintains a **hidden state** $\mathbf{h}_t$ — a running summary of everything seen so far.

At each step it does two things: update the summary using the new input *and* the previous summary, then optionally emit an output. The same weights are used at every step, which is weight sharing again — along time rather than along space.

The consequence worth sitting with. There is no fixed window. In principle $\mathbf{h}_t$ encodes the entire history $x_1, \ldots, x_t$, so the architecture imposes no limit on how far back the model can look.

In principle. Most of this lecture is about the gap between that promise and what a recurrent network actually delivers — and about the architecture built to close it.

The Vanilla RNN

One equation, applied over and over — and the reason that is both the whole idea and the whole problem.

Vanilla RNN: One Equation, Repeated

The recurrence

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}), \qquad \hat{y}_t = \mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y$$

$\mathbf{W}_h$, $\mathbf{W}_x$, $\mathbf{W}_y$ are the *same* matrices at every step.

Read the first equation in words: the new summary is a squashed mix of the old summary and the new observation. That is the entire model.

Because the weights are shared across steps, a 26-week window costs no more parameters than a 5-week one — and the same network handles sequences of different lengths without modification. Our RNN below holds about **4,500 parameters**, against 22,657 for the small feedforward network in Lecture 7.

For forecasting we usually want one number, not a sequence, so we take the **final** hidden state $\mathbf{h}_T$ — the summary after reading the whole window — and put one linear layer on top of it.

Training It: Backpropagation Through Time

Training an RNN is ordinary backpropagation applied to an unusual graph. **Unroll** the recurrence across the window and it becomes a 26-layer feedforward network — one layer per time step — in which every layer shares the same weights.

Gradients then flow backward from the loss at step T all the way to step 1. Because $\mathbf{W}_h$ appears at every step, its gradient is a *sum* of contributions from all 26 positions.

> **This is where it breaks.** Propagating a gradient back through k steps multiplies by roughly $\mathbf{W}_h$ k times. If the relevant factor is below 1, the gradient shrinks *exponentially*; above 1, it explodes. Either way, long-range signal does not survive the trip.

Exploding gradients have an easy fix — `clip_grad_norm_` caps the size of the update, and we use it. Vanishing gradients do not, which is what the next section is for.

Why a Vanilla RNN Forgets (Bengio et al. 1994)

Put numbers on it. Suppose each backward step multiplies the gradient by a factor of 0.7 — an unremarkable value for a $\tanh$ network.

Steps back	Gradient factor	Meaning
1	0.70	full signal
5	0.17	weakened but usable
10	0.028	barely present
26	$0.000\,002$	gone
52	$\approx 10^{-9}$	the annual lag, invisible

The promise from two slides ago fails in practice. A vanilla RNN *can* in principle encode arbitrarily long history, but the gradient that would teach it to do so has vanished long before it reaches back that far. In practice these networks learn dependencies of roughly ten steps.

Bengio and colleagues proved this is not an optimization bug to be tuned away — it is inherent to the architecture. Which is why the fix had to be a different architecture.

LSTM: Long Short-Term Memory

Add a memory that information passes through rather than being recomputed at every step.

LSTM: A Cell State and Three Gates (Hochreiter and Schmidhuber 1997)

An LSTM carries **two** things forward: a **cell state** $\mathbf{c}_t$, which is the long-term memory, and a **hidden state** $\mathbf{h}_t$, which is what the rest of the network sees. Three gates control the traffic between them.

Forget gate	$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$ — what fraction of the old memory to keep
Input gate	$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$ — how much of the new candidate to write
Output gate	$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$ — how much of memory to expose

The memory update itself is the line that matters:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} \; + \; \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Each gate is a sigmoid, so it outputs between 0 and 1 — a soft switch the network learns.

What the Gates Are Doing, in Plain Terms

The equations are compact but the behavior is intuitive. Take a store forecasting demand week by week.

Forget	"The promotion that ran three months ago no longer explains anything — drop it." The gate closes toward 0 and that part of memory decays away.
Input	"This week is Thanksgiving and the spike is enormous — record it." The gate opens toward 1 and the event is written into the cell.
Output	"I am holding a memory of last December, but this week is April — do not act on it yet." Information stays in $\mathbf{c}_t$ without contaminating the current prediction.

The output gate is the one people skip, and it is the subtle one. It separates *what the model remembers* from *what the model uses right now*. A vanilla RNN cannot make that distinction — its single hidden state has to serve both jobs at once, so anything it stores is immediately in play.

Why the Cell State Fixes the Gradient

Look again at the memory update, and specifically at the operation in the middle:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

The old memory enters **additively**, and it is not passed through a weight matrix or a $\tanh$ on the way. When the forget gate is near 1, $\mathbf{c}_t \approx \mathbf{c}_{t-1}$ plus whatever is new — and the gradient flows backward through that addition essentially undiminished.

Compare with the vanilla RNN, where every backward step multiplies by $\mathbf{W}_h$ and the $\tanh$ derivative. Repeated multiplication shrinks; repeated addition does not.

> **You have seen this fix before.** It is the ResNet skip connection from Lecture 8 — $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$ — solving the same problem in the same way. Give the gradient an additive path home and depth stops destroying it.

LSTM (1997) predates ResNet (2015) by eighteen years — the idea was rediscovered for depth after being invented for time.

In PyTorch, and Does It Work

Four lines of model code, and an honest measurement against everything else this course has fitted.

LSTM for Forecasting in PyTorch

Window in, one forecast out

```
class LSTMForecaster(nn.Module):
def __init__(self, n_features, hidden=64, layers=1):
super().__init__()
self.lstm = nn.LSTM(n_features, hidden, num_layers=layers,
batch_first=True)
self.fc = nn.Linear(hidden, 1)
def forward(self, x):      # x: (batch, seq_len, n_features)
out, (h, c) = self.lstm(x)
return self.fc(out[:, -1, :]).squeeze(-1)
```

`batch_first=True` gives shape `(batch, seq_len, features)` — the *opposite* of `Conv1d`. `out[:, -1, :]` is the state after the last step. Lecture 7's rules still hold, plus `clip_grad_norm_(..., 1.0)`.

Does It Work? The Scoreboard So Far

Same weekly panel, same 26-week windows, same test block as Homework 4.

Model	Test RMSE	Parameters
LASSO, 46 engineered features	**744**	46
XGBoost, 46 engineered features	781	—
Vanilla RNN, 4 raw channels	**842**	4,545
Random forest	899	—
LSTM, 4 raw channels	987	17,985
1D CNN, best variant	1,032	8,161
Seasonal naive	2,152	0

Recurrence beats convolution decisively — and it did so on *four raw channels* with no lag engineering at all, against 46 hand-built features for LASSO. That is the case for reading a sequence in order, in one line.

And the LSTM lost to the vanilla RNN — at a fixed 30-epoch budget. That is not a typo, and the next three slides are about what it does and does not mean.

Why the Gated Model Lost at 30 Epochs

The result holds at all three window lengths — but all three share one panel and one split, so this is one finding measured three ways, not three independent tests:

Window	Vanilla RNN	LSTM	
13 weeks	**878**	990	RNN wins
26 weeks	**842**	987	RNN wins
52 weeks	**805**	998	RNN wins

An LSTM solves a problem this data does not have — gating preserves information across *long* chains, and weekly demand is dominated by the recent past. The gates are not free: **17,985 parameters against 4,545**. Before concluding they bought nothing, this table owes two checks: **was the comparison fair**, and **what in the data explains it**.

> A margin measured at one training budget is a claim about that budget. 30 epochs was the default, not a setting shown to be enough for both models.

Check 1: Was It a Fair Fight?

An LSTM carries four times the parameters. Bigger models converge slower. So vary the **training budget** and watch the margin, at the 26-week window:

Epochs	**10**	**30**	**60**	**100**
Vanilla RNN	1,099	**842**	828	830
LSTM	1,002	987	926	**821**
Gap (LSTM − RNN)	-97	$+145$	$+98$	-9

The gap is **largest at exactly the budget we used**, and it closes at every checkpoint after it. The RNN is flat from about epoch 45; the LSTM is *still improving* at 100 — we ran out of patience, not the model.

At a fixed 30-epoch budget the RNN wins by a wide margin. At convergence the two **tie**. The RNN's genuine advantage is that it gets there roughly **three times faster on a quarter of the parameters** — an optimization advantage, not a verdict on gating.

Check 2: What in the Data Explains It

Gating pays when a value must survive many steps that would otherwise overwrite it. Three measurements say this panel has no such structure.

1. Most variance is not sequential. 91% of the variation in weekly units is *between* series — which store, which category. The main task is reading the level, and the last few weeks give you that.

2. The series are smooth. Within-series autocorrelation is **0.83** at lag 1 and **0.43** at lag 52 — so the recent past is a *sufficient statistic* for the distant past. Nothing must be carried across a gap, because the gap was filled in continuously.

3. Nothing arrives early. `snap_days` correlates $+0.14$ with units in the same week and ≈ 0 at every lead; `event_days` is flat everywhere. No *announcement* structure — precisely what a forget gate is for.

> Ask these of your own series before reaching for a gated model: how much of my variance is cross-sectional? Do recent values already summarize old ones? Does anything in my inputs **lead** the target?

Key Takeaways

What recurrence buys, what it costs, and what is still missing.

RNN and LSTM in Context

Architecture	Sees order?	Effective memory	Cost
FFN on lags	no	whatever you engineered	cheap
1D CNN	local only	receptive field ($\sim$7 steps)	cheap
Vanilla RNN	yes	$\sim$10–30 steps in practice	cheap
LSTM	yes	hundreds of steps	4$\times$ an RNN

An LSTM holds roughly **four times** the parameters of a vanilla RNN at the same hidden size — each of the three gates plus the candidate needs its own weight matrix. That is the price of the memory, and on a short window you pay it for nothing.

What is still missing. An LSTM reads strictly left to right, one step at a time, so it cannot be parallelized across the sequence, and everything it knows about step 3 must survive being squeezed through every state between 3 and T. Part 2 removes both constraints.

Lecture 9 Part 1: Key Takeaways

1. A **recurrent network** reads a sequence in order and carries a hidden state. Weights are shared across time, so window length costs no parameters.
2. Training is **backpropagation through time**: unroll the recurrence and it is a deep network whose layers share weights. Repeated multiplication shrinks the gradient exponentially — at 52 steps, around 10^{-9}.
3. An **LSTM** adds a cell state updated **additively**. Addition preserves gradients where multiplication destroys them — the ResNet fix, eighteen years earlier.
4. Three gates: **forget** what is stale, **input** what is new, **output** what is relevant now.
5. On our panel both recurrent models **beat every CNN variant** on four raw channels — and both still lose to a 46-coefficient LASSO.
6. The **LSTM lost to the vanilla RNN at a 30-epoch budget**, at all three window lengths — but the two **tie** once both reach convergence, and 91% of this panel's variance is cross-sectional anyway. **Fit the architecture to the problem — and check a margin survives the budget you measured it at.**

References I

- Bengio, Yoshua, Patrice Simard, and Paolo Frasconi (1994). "Learning Long-Term Dependencies with Gradient Descent Is Difficult". In: *IEEE Transactions on Neural Networks* 5.2, pp. 157–166.
- Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9.8, pp. 1735–1780.